

Documentação: Autenticação via Google OAuth 2.0

Justificativa Tecnológica

Optar pelo protocolo OAuth 2.0 mitiga graves vulnerabilidades de segurança. Como a aplicação não armazena, não processa e não trafega senhas dos usuários, o risco de vazamentos em caso de falha no banco de dados é reduzido drasticamente. O controle de acesso interno passa a ser feito puramente por credenciais baseadas em JWT (JSON Web Tokens).

Fluxo de funcionamento

- O cliente (interface web) dispara o *prompt* oficial do Google pedindo permissão de acesso ao usuário.
- Com a autorização concedida, o Google emite um Token de Identidade (*ID Token*) assinado criptograficamente e o devolve para o front-end.
- O front-end envia esse token para a rota de conversão do nosso servidor.
- O back-end recebe a carga e valida a integridade da assinatura usando as chaves públicas oficiais do Google. Se houver divergência ou expiração, o processo morre aqui com erro `400`.
- Com a assinatura confirmada, o sistema extrai o e-mail e valida se o usuário já possui cadastro. Usuários novos são criados automaticamente.
- O evento de login dispara um gatilho de "auditoria e registro (log) dos acessos e das ações dos usuários", salvando a data, hora e IP de origem.
- O sistema finaliza o processo gerando um *Access Token* e um *Refresh Token* locais e os devolve ao cliente.

Nossa aplicação cliente nunca possui as credenciais de servidor. Toda a configuração secreta (*Client Secret*) reside unicamente nas variáveis de ambiente do back-end.

Persistência de Dados

O módulo impacta as seguintes tabelas no banco de dados relacional:

Entidade	Papel no Sistema	Cardinalidade
Account / Usuario	Armazena o e-mail, nome, permissões e a data do consentimento dos termos de uso.	Tabela primária.
AuditLog / LogAcesso	Cumpr o requisito de manter o "registro (log) dos acessos" da aplicação.	

| 1:N (Um usuário para muitos logs). |

Rotas de Integração (API)

A disponibilização do serviço foi estruturada utilizando o Django REST framework. Para consumir o login social, as seguintes interfaces foram expostas:

Método	Endpoint	Proteção	Objetivo
POST	/api/v1/identity/google/	Pública	Recebe o <code>id_token</code> do Google, processa a auditoria e devolve o JWT da aplicação.
POST	/api/v1/identity/refresh/	Pública	Permite renovar a sessão do cliente utilizando o token de atualização sem exigir novo login no Google.

Respostas, Falhas e Tratamento de Exceções

As requisições passam por uma esteira de validação que retorna os seguintes códigos HTTP:

Status HTTP	Causa	Resultado
200 OK	Assinatura válida e cadastro ativo.	Retorna o payload com os tokens JWT locais.
400 Bad Request	Token do Google malformado, expirado ou forjado.	Conexão recusada preventivamente, impedindo acesso ao banco.
403 Forbidden	Usuário pendente de aceite legal (ex: termos não assinados).	Exige que o front-end redirecione para o formulário de concordância.
403 Forbidden	Perfil banido ou inativado no painel administrativo.	Acesso negado com mensagem de conta suspensa.

Conformidade Legal e Privacidade

A captação de dados foi arquitetada considerando rigorosamente as "informações sobre a LGPD, incluindo termo de uso e política de privacidade" exigidas no projeto.

- Minimização:** O escopo solicitado ao Google restringe-se apenas a `email` e `profile`. Nenhuma informação da conta pessoal (como drive ou contatos) é acessada.

Parametrização do Ambiente

Para inicializar o serviço, o sistema exige que as variáveis de ambiente abaixo estejam preenchidas no arquivo `.env`:

```
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
```

Sem essas variáveis, o servidor levanta, mas as rotas descritas no tópico de API abortam as requisições devolvendo um erro interno `500` por falha de configuração.